

DESIGN AND ANALYSIS OF ALGORITHMS

Complete Solutions Reference Manual (Short & Easy Answers)

Part A: Multiple Choice Questions

1. Merge Sort follows which strategy?

Correct Answer: iii) Divide and Conquer

Explanation: Merge sort breaks the array into two halves, recursively sorts them, and then merges the sorted halves.

2. Which algorithm is used to find the shortest path in a weighted graph?

Correct Answer: iii) Dijkstra's Algorithm

Explanation: Dijkstra's algorithm finds the single-source shortest path for graphs with non-negative edge weights. Prim's and Kruskal's are for Minimum Spanning Trees.

3. Which of the following approaches is Divide and Conquer strategy?

Correct Answer: ii) Bottom-up

Explanation: Though divide and conquer is logically top-down in dividing, the actual solution building and combining phase moves from the base cases upward (bottom-up execution). Alternatively, if viewed as structured recursion, some contexts consider it top-down, but in standard classification of algorithmic subproblem resolution, merging operates bottom-up.

4. The average number of comparisons performed by merge sort algorithm in merging two sorted lists of length 2 is _____.

Correct Answer: 2.67 (or 3 in the worst case)

Explanation: To merge two lists of size 2, the number of comparisons can range from 2 (best case: e.g., [1,2] and [3,4]) to 3 (worst case: e.g., [1,3] and [2,4]). Mathematically, the exact average across all permutations is 2.67.

5. Which of the following sorting algorithms has the best average-case time complexity?

Correct Answer: iii) Merge Sort

Explanation: Merge Sort has an average-case complexity of $O(n \log n)$, whereas Bubble, Selection, and Insertion sort all have $O(n^2)$ average-case complexities.

6. Solve the following recurrence relation using Master's theorem: $T(n) = 3T(n/2) + n^2$

Correct Answer: $\Theta(n^2)$

Explanation: Here, $a = 3$, $b = 2$, $f(n) = n^2$. $n^{\log_b a} = n^{\log_2 3} \approx n^{1.58}$. Since $f(n) = n^2$ grows polynomially faster than $n^{1.58}$ (Case 3 of Master's Theorem), $T(n) = \Theta(f(n)) = \Theta(n^2)$.

7. Which of the following algorithm design techniques is used in Quick Sort algorithm?

Correct Answer: iii) Divide and Conquer

Explanation: Quick Sort partitions the array around a pivot element (divide) and then recursively sorts the subarrays (conquer).

8. Which of the following problems can be solved using Dynamic Programming?

Correct Answer: iii) Fibonacci Series

Explanation: Fibonacci series exhibits overlapping subproblems, making it a classic candidate for Dynamic Programming. Fractional Knapsack uses a Greedy approach, and Tower of Hanoi uses pure Divide & Conquer.

9. Which case of Master's Theorem applies for $T(n) = 2T(n/2) + n$?

Correct Answer: ii) Case 2

Explanation: Here $a = 2$, $b = 2$, so $n^{\log_b a} = n^{\log_2 2} = n^1 = n$. Since $f(n) = n$ is equal to $n^{\log_b a}$, it falls directly under Case 2, giving $T(n) = \Theta(n \log n)$.

10. Which algorithm is used for finding Minimum Spanning Tree?

Correct Answer: ii) Prim's Algorithm

Explanation: Prim's and Kruskal's algorithms are standard greedy algorithms used to find a Minimum Spanning Tree (MST). Floyd-Warshall and Bellman-Ford are shortest path algorithms.

Part B: Descriptive Questions

1. Define different asymptotic notation O , Ω , Θ with suitable examples.

- **Big-O (O) - Upper Bound:** Defines the worst-case running time. $f(n) = O(g(n))$ if there exist constants c , $n_0 > 0$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
Example: $2n + 3 = O(n)$.
- **Big-Omega (Ω) - Lower Bound:** Defines the best-case running time. $f(n) = \Omega(g(n))$ if there exist constants c , $n_0 > 0$ such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.
Example: $3n^2 + 5 = \Omega(n^2)$.
- **Big-Theta (Θ) - Tight Bound:** Defines the average-case tight behavior. $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
Example: $5n = \Theta(n)$.

2. Explain the Divide and Conquer strategy with a suitable example. Discuss its advantages and disadvantages.

Strategy: It works by breaking a complex problem into smaller subproblems of the same type (Divide), solving them recursively (Conquer), and then combining their solutions to solve the original problem (Combine).

Example: Merge Sort divides an array into single elements, sorts them, and merges them back.

Advantages: Solves difficult problems efficiently; parallelizes easily; balances workload.

Disadvantages: Overhead of high recursion stack depth; may use extra space (like Merge Sort).

3. Derive the recurrence relation for the Merge Sort algorithm.

Let $T(n)$ be the time required to sort an array of size n .

1. **Divide:** Splitting the array takes constant time: $O(1)$.
2. **Conquer:** Recursively solving two subarrays of size $n/2$ takes: $2T(n/2)$.
3. **Combine:** Merging two sorted lists of total size n takes: $O(n)$.

Summing these up gives the standard recurrence relation:

$$T(n) = 2T(n/2) + cn \text{ for } n > 1, \text{ and } T(1) = 1$$

4. Given the weight vector (2, 3, 5, 7, 1, 4, 1) and profit vector (10, 5, 15, 7, 6, 18, 3) and a knapsack of capacity 15, find a feasible solution. Discuss whether it is optimal or not.

Assuming a **Fractional Knapsack** problem (standard for greedy weight/profit vectors):

Calculate the profit-to-weight ratio (p_i/w_i) for each item:

- I1: $10/2 = 5.0$ | I2: $5/3 = 1.67$ | I3: $15/5 = 3.0$ | I4: $7/7 = 1.0$ | I5: $6/1 = 6.0$ | I6: $18/4 = 4.5$ | I7: $3/1 = 3.0$

Sort items by ratio descending: **I5 (6.0), I1 (5.0), I6 (4.5), I3 (3.0), I7 (3.0), I2 (1.67), I4 (1.0).**

Greedy Selection (Capacity = 15):

1. Take full **I5**: weight = 1, profit = 6 (Remaining Capacity = 14)
2. Take full **I1**: weight = 2, profit = 10 (Remaining Capacity = 12)
3. Take full **I6**: weight = 4, profit = 18 (Remaining Capacity = 8)
4. Take full **I3**: weight = 5, profit = 15 (Remaining Capacity = 3)
5. Take full **I7**: weight = 1, profit = 3 (Remaining Capacity = 2)
6. Take fraction of **I2**: take 2 units out of 3. Profit = $(2/3) \times 5 = 3.33$ (Remaining Capacity = 0)

Feasible Solution Vector (x): (1, 0.67, 1, 0, 1, 1, 1)

Total Profit: $6 + 10 + 18 + 15 + 3 + 3.33 = 55.33$

Optimality: Yes, this greedy choice is guaranteed to yield the **optimal solution** for the Fractional Knapsack problem.

5. Compare Merge Sort and Quick Sort based on: Time complexity, Space complexity, Stability, Method used.

Feature	Merge Sort	Quick Sort
Time Complexity	Best/Avg/Worst: $O(n \log n)$	Best/Avg: $O(n \log n)$, Worst: $O(n^2)$
Space Complexity	$O(n)$ (Extra space for merging)	$O(\log n)$ (In-place recursion stack)
Stability	Stable (Preserves original order)	Unstable
Method Used	Divide, conquer, and external merge	Divide (partition around pivot) and conquer

6. Find the time complexity of the following:

(a) $T(n) = 8T(n/2) + n^3$

(b) $T(n) = 16T(n/4) + n^2$

Using Master's Theorem:

- (a) $T(n) = 8T(n/2) + n^3$: $a=8$, $b=2 \rightarrow n^{\log_2 8} = n^3$. Since $f(n) = n^3$ matches n^3 , this is Case 2.
Complexity: $\Theta(n^3 \log n)$
- (b) $T(n) = 16T(n/4) + n^2$: $a=16$, $b=4 \rightarrow n^{\log_4 16} = n^2$. Since $f(n) = n^2$ matches n^2 , this is Case 2.
Complexity: $\Theta(n^2 \log n)$

7. Explain the concept of recursion. Write a recursive algorithm for finding factorial of a number and analyze its complexity.

Concept: Recursion is a process where a function calls itself directly or indirectly to solve a smaller instance of the same problem until it reaches a defined base case.

Algorithm:

```
function Factorial(n):
    if n == 0 or n == 1:
        return 1          // Base case
    else:
        return n * Factorial(n - 1)
```

Complexity Analysis: The recurrence relation is $T(n) = T(n-1) + O(1)$. Expanding this yields $T(n) = T(n-2) + 2 = \dots = O(n)$.

Time Complexity: $O(n)$ | **Space Complexity:** $O(n)$ (due to recursion stack frame).

8. Explain the working principle of Binary Search with a suitable example. Derive its time complexity.

Principle: Works only on a **sorted array**. It repeatedly divides the search interval in half. It compares the target element with the middle element. If they match, the position is returned. If the target is smaller, it

searches the left half; if larger, it searches the right half.

Example: Search for 7 in [2, 4, 5, 7, 9]. Mid is 5. Since $7 > 5$, search the right sub-array [7, 9]. New mid is 7. Found!

Derivation: Each step cuts the array size in half: $n, n/2, n/4, \dots, 1$.

Let k be the number of steps: $n / 2^k = 1 \rightarrow n = 2^k \rightarrow k = \log_2 n$.

Time Complexity: $O(\log n)$.

9. Describe the characteristics of Greedy Method. Critically comment on..... Greedy strategy does not work for the 0-1 knapsack problem for all time.

Characteristics: Feasible solution selection, local optimal choice at every step, irrevocable decisions (never reconsiders choices).

Commentary on 0-1 Knapsack: In a 0-1 Knapsack, items cannot be broken; you must either take an item full or leave it. The greedy strategy (taking items with highest profit/weight ratio) fails because a greedy choice can leave large empty spaces in the knapsack, leading to sub-optimal total profit.

Counter-example: Knapsack capacity = 10. Item A ($w=6, p=12, \text{ratio}=2$), Item B ($w=5, p=9, \text{ratio}=1.8$), Item C ($w=5, p=9, \text{ratio}=1.8$).

Greedy chooses Item A (leaves space 4, total profit=12). But optimal choice is Items B and C (total weight=10, total profit=18). Thus, greedy does not work for 0-1 knapsack.

10. Discuss the importance of algorithm design techniques in problem solving.

Algorithm design techniques (like Greedy, Dynamic Programming, Divide & Conquer) provide structured blueprints to solve complex computations. Their importance includes:

- **Efficiency:** Converts naive exponential-time solutions ($O(2^n)$) into polynomial-time solutions ($O(n \log n)$ or $O(n^2)$).
- **Resource Optimization:** Saves physical computing hardware infrastructure by reducing execution time and memory footprints.
- **Scalability:** Ensures software applications run reliably well when data scales from kilobytes to petabytes.
- **Reusability:** Common mathematical frameworks can be applied across different fields (e.g., dynamic programming works for DNA sequencing as well as logistics budgeting).